Regulatory Activity of DNA – Martyna Pawlak

1. Introduction and purpose of the project

The main goal of the project is to develop a machine learning model to see the relationship between DNA sequence and regulatory activity. Described task was defined as multi task learning in which the model must focus on two goals:

- is_active - whether it is an active sequence or not (0 or 1)
- rna_dna_ratio - continuous value representing regulatory strength

Predicting DNA activity based on sequence is difficult, because it also depends (among other things) on the combination of motifs and their mutual arrangements.

On the more technical side: I uploaded the files on GitHub and then trained the model on Google Colab.

2. Data set analysis

Our input data are nucleotide sequences of a specific length. One-hot encoding was used to allow the model to process and understand them. This will ensure that each letter is equidistant from the others. Each nucleotide is converted into a four-element vector:

A -> [1,0,0,0]
T -> [0,1,0,0]
C -> [0,0,1,0]
G -> [0,0,0,1]

3. Model architecture (methodology)

Applied model is a CNN type network. The architecture is based on two independent output heads: one to predict classification (0 or 1 value) and the other one to predict regression (continuous value).

- Convolutional layers

The model is composed of two convolutional layers. After numerous tests, this arrangement was found to be the most optimal. First layer (conv1) has 64 filters, each 21 nucleotides wide. This large kernel size allows for the capture of long transcription factor binding motifs. Second layer (conv2) has 128 filters, each 11 nucleotides wide. Reducing the kernel size allows for the detection of more complex combinations of previously identified motifs.

- Output heads

After flattening the data, the model splits into two paths:

Task	**Layer**	**Activation function**	**Output**
Classification (is_active)	Linear(128,1)	Sigmoid	Probability (0,1)
Regression (rna_dna_ratio)	Linear(128,1)	None (linear)	Continuous value

In the classification head, sigmoid function computes the probability whether the sequence is active or not. The final membership in the binary class (is_active) is determined based on the decision threshold (in this model threshold = 0.5). If the result > 0.5, then the sequence is described as active.

- Data augmentation

In order to increase the model's resistance to the specificity of the sequence recording, the reverse complement method was used for augmentation. It involves reversing the order of nucleotides and replacing them with complementary ones. Thanks to reverse complement, we create new examples for the model to learn and it shows that it learns better – it is described more precisely later in the report.

4. Learning process (training)

The key task was to create an architecture that would allow the model to optimize two different goals: classification and regression. For this purpose, a loss function that combines these two criteria was implemented.

- **Binary cross entropy (BCE) –** used for classification head
- **Mean squared error (MSE) –** used for regression head

An important element of the training is the weighting of the elements of the loss function. The mse values may be larger than those of bce, so summing them directly could result in the model only focusing on regression improvement, ignoring classification. To solve this problem, the formula was used:

$$Loss_total = loss_class + 0.1 * loss_reg$$

5. Results

The final model was trained on 100 epochs. The graph below shows the loss function for the training dataset and validation dataset:

We can observe that both of the curves are decrasing and they stay close to each other throughout the training proces. This proves that we are not dealing with overfitting and the model is actually learning biological systems.

- **Classification task (is_active) -** the model achieved a result that was approximately 6 percentage points higher than the baseline (from 67.4 to 73.1)

- **Regression task (rna_dna_ratio) –** the model increased the value from 0.0441 to 0.2389. Although a score of 0.23 may seem low in standard tasks, in molecular biology we can consider this a solid result.

6. Discussion and conclusions (experiments)

During the model development process, I performed numerous tests to determine which parameters and configurations would be most optimal. Below are two of the most interesting observations.

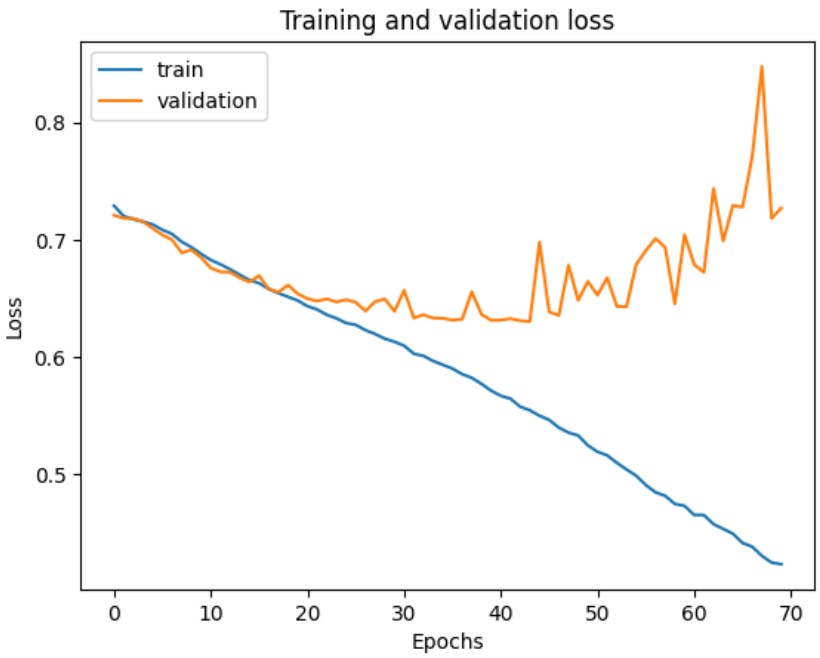
- **Third layer**

To test whether even higher precision could be achieved I added a third convolutional layer. The hypothesis was that a deeper network would better capture biological relationships.

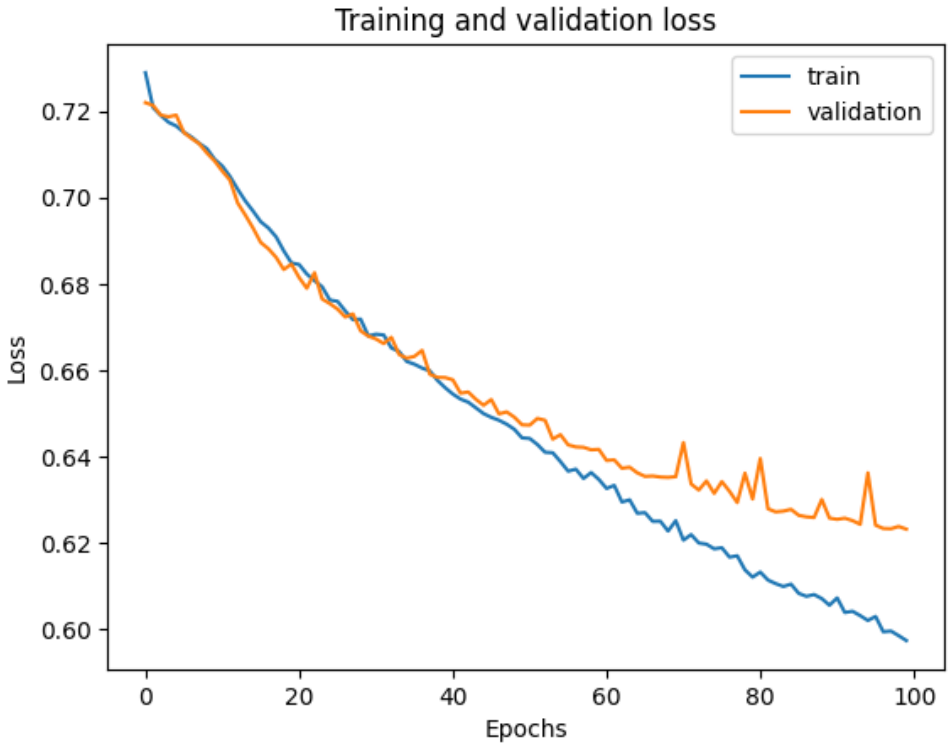
```
self.conv3 = nn.Conv1d(128,256,7, padding='same')
self.relu3 = nn.ReLU()
self.pool3 = nn.MaxPool1d(2)
self.drop3 = nn.Dropout1d(0.3)
```

```
x = self.conv3(x)
x = self.relu3(x)
x = self.pool3(x)
x = self.drop3(x)
```

With the addition of the third layer, a large overfitting was revealed. Training loss was steadily decreasing, while validation loss began to increase rapidly. I trained it on only 70 epochs, because with 100 epochs result would be the same.



- **Without augmentation**

I also tried to determine if augmentation would be a good choice for improving the model. It turned out to be. It might be a small difference in optimality, but on a scale of 100 epochs we can see that the model without augmentation shows a slight overfitting. Therefore, I ultimately chose the augmented model – there was no overfitting, and the accuracy score was a little bit higher.

```
train_dataset = DNADataset(train_path, mean_reg, std_reg, augment=False)
val_dataset = DNADataset(val_path, mean_reg, std_reg, augment=False)
```